

看板驅動的 Agentic Loop： 金融級安全 AI Agent 研發新範式

以「唯讀憲法」構築跨團隊安全護欄，打造人控方向、AI 奔跑的自動化閉環

移卡即變現：看板操作觸發 Agentic Loop 閉環運轉，在金融合規護欄下，人只負責定方向，其餘交給 AI。

01 情境 — 一個動作，啟動整個開發迴圈

PM 在看板工具（Taiga／Jira／Jenkins…）把卡片從「Backlog」拖到「In Dev」，**不需再做任何事**——AI agent 自動讀取卡片上的需求描述、開始程式開發、跑測試、對照安全規則，最後產出一個已審查、合規的 PR，等候工程師確認合併。

QA Agent 自動依驗收情境跑測試。整個跨職能團隊——PM、工程師——都能以**自然的工作動作**驅動 AI 開發迴圈，不再需要手動啟動任何腳本。

02 痛點 — 放手自動迭代的三個風險

時間 時間黑洞。有了好規格與憲法，一致性問題解決了，但每一輪「跑測試 → 看錯誤 → 修改 → 再複查」仍要**手動**進行，這才是真正吞掉時間的地方。

安全 安全風險。在銀行裡若放手讓 AI 無限制自跑很危險——它可能為了讓測試通過而偷偷刪掉測試（reward hacking，鑽漏洞取巧），或在不知不覺中違反某條安全規則。

流程 流程斷點。agent 迴圈與日常敏捷流程脫節——團隊各自在看板推卡片，卻要另外手動啟動 agent，造成斷點，也讓非工程師角色難以參與。

03 解法 — 一個受控的 Actor ↔ Critic 迴圈



看板四欄（Backlog → Spec → Dev → PR/Done）由左至右流動，移卡即觸發 Agentic Loop，Spec 與合併兩端各設一道人工把關。

①

受控封閉迴圈

Actor ↔ Critic 固定執行 6 次，驗證機制對 Actor **唯讀、不可竄改**。

②

兩道人工把關

Spec 交 Actor 前審查需求方向；合併前人工 Review & Merge。

③

可維護的憲法

版本控管的規則集（spec-driven），新增一條即套用至後續所有變更，跨團隊可沿用。

工具 Codex + 自訂 GPT（Spec/QA/Security）+ Connector 串接看板工具與 CI。

04 預期效益

自動運轉

把手動的「寫 → 測 → 修 → 複查」變成能自動運轉的 agent 迴圈，直接產出一個已審查、合規、通過安全檢核的 PR。

安全實證

「設上限 + 可驗證 + 人工把關」的迴圈紀律已設計進 App 的 orchestrator——以此 App 作為實證，證明團隊交付的是**安全可控的 agent**，而非失控的迴圈。

可擴散

可重用的迴圈樣板、護欄與可維護的憲法，能擴散到集團所有開發團隊。

3 句話總結

- 1 在看板移一張 task，AI 就自動完成程式開發、測試、安全複查，產出合規 PR——但在銀行裡直接放手讓 AI 無限制自跑很危險：它可能取巧改掉測試，或悄悄違反安全規則。
- 2 我們做的是「受控迴圈」：看板卡片流轉觸發 Actor↔Critic，驗證機制對 Actor 唯讀不可竄改、設有迭代上限、規格與合併兩端都保留人工把關——讓整個跨職能團隊都能以自然動作驅動 AI，且我們已用同樣紀律打造出這個 AI-Native 銀行 App 作為實證。
- 3 預期能把整段手動迭代自動化、產出合規且通過安全檢核的程式碼，並把「安全的 agent 迴圈」與「可維護的憲法」樣板擴散到集團各團隊。